Shopping With Coupons

Date: 2026-11-11

Abstract

We have N items, N kinds of coupons and D dollars, and we want to buy as many items as we can. Our idea is greedy: always buy the cheapest (item, coupon) pair that is still left. We wrote three programs for this idea: brute force, a heap method and a binary search method. They give the same answers in all our tests, and the two fast ones finish the largest inputs in about 30 ms.

Contents

1 Introduction **3**

2 Algorithm Specification **3**

2.1 Greedy Idea . . . 3

2.2 Brute Force . . . 3

2.3 Heap Method . . . 4

2.4 Binary Search Method . . . 4

2.5 Correctness . . . 5

3 Testing Results **5**

3.1 Method . . . 5

3.2 Correctness Tests . . . 6

3.3 Test Data . . . 6

3.4 Timing Results . . . 7

4 Analysis and Comments **8**

4.1 Time Complexity . . . 8

4.2 Space Complexity . . . 9

4.3 Discussion of Test Results . . . 9

4.4 Limitations and Improvements . . . 9

5 Conclusion **9**

6 References **10**

7 Source Code Appendix **10**

1 Introduction

In this problem there are N items on a shopping list and N kinds of coupons, and we have D dollars. We can buy an item many times and use a coupon many times, but one coupon can be used on one item only once. We want to buy as many items as possible. If there are several ways to do that, we want the one with the most money left.

Let us look at the example in the problem. There are 4 items with prices 10, 12, 15 and 20, and 4 coupons with values 6, 7, 8 and 9, and we have 30 dollars. The cheapest thing we can do is to buy the 10-dollar item with the 9-dollar coupon, which costs only 1 dollar. We can buy the 10-dollar item again with the coupons 8, 7 and 6, which costs 2, 3 and 4 dollars. Then we buy the 12-dollar item with the coupons 9, 8 and 7 for 3, 4 and 5 dollars, and the 15-dollar item with the coupon 9 for 6 dollars. In total we pay 28 dollars for 8 items and have 2 dollars left, which is not enough for anything else.

From this example we can see what the problem is really about. Every pair of one item and one coupon is a possible purchase, its cost is the price minus the coupon, and each pair can be used only once. We just need to pick the cheapest pairs. The problem promises that the largest coupon is smaller than the lowest price, so every pair costs at least 1 dollar.

The input has N ($\leq 10^5$) and D ($\leq 10^6$) on the first line, then the N prices, then the N coupon values. We print the number of items and the money left.

The hard part is that there are too many pairs. When $N = 10^5$ there are 10^{10} pairs, so we can not write them all down and sort them. But we also do not need to: with at most 10^6 dollars and at least 1 dollar per item, we never buy more than 10^6 items. So we only need a fast way to find the cheapest pairs.

2 Algorithm Specification

2.1 Greedy Idea

All our methods use the same rule: buy the cheapest pair that is not used yet, and stop when we can not pay for it. This works because buying one pair does not change the cost of any other pair, so there is no reason to skip a cheap pair and take an expensive one. The three methods below only differ in how they find the cheapest pair.

2.2 Brute Force

The most direct way is to compute the cost of all N^2 pairs, sort them, and buy from the cheapest one until the money runs out. It is simple and hard to get wrong, so we use it to check the other methods. But it needs N^2 numbers in memory, so we only allow it when $N \leq 5000$.

2.3 Heap Method

To avoid looking at all pairs, we first sort the prices from low to high and the coupons from high to low. Now for one item, its cheapest pair uses the largest coupon, its second cheapest pair uses the second largest coupon, and so on. So each item has a list of pairs that go from cheap to expensive, and we only need to look at the first unused pair of each item. Also, a more expensive item can not have the cheapest pair before the cheaper item has used its largest coupon, so we do not need to look at all items from the start.

We keep the first unused pair of each item in a min-heap, because a min-heap can give us the smallest element quickly [1]. The steps are:

1. **Start.** Put the pair (cheapest item, largest coupon) into the heap.
2. **Buy.** Take the cheapest pair out of the heap. If we can not pay for it, stop. Otherwise pay and count one more item.
3. **Add.** Put the next pair of the same item (with the next coupon) into the heap. If we just used the largest coupon on this item, also put the first pair of the next item into the heap. Then go back to step 2.

In the example, the heap gives the costs 1, 2, 3, 3, 4, 4, 5, 6. Their sum is 28. The next pair costs 6 but we only have 2 dollars, so we stop with 8 items and 2 dollars left.

2.4 Binary Search Method

The heap method takes one pair at a time. If every pair is very cheap, it has to do this up to 10^6 times. So we also tried another way that does not go through the pairs one by one.

The idea is to guess a cost t and ask: if we buy every pair that costs at most t, how much do we pay? If this is not more than D, then t can be larger; if it is more than D, then t is too large. So we can find the best t by binary search.

1. **Count for one t.** For an item with price p, the pairs that cost at most t are the ones with a coupon of at least $p - t$. The coupons are sorted, so these are the first k coupons. We get their total cost from k, p and a prefix sum of the coupons. For the next item the price is higher, so k can only become smaller. This means we just move one pointer backwards and finish all items in one pass.
2. **Binary search.** Find the largest t for which the total is at most D.
3. **Use the rest.** After buying all pairs up to t, there is usually some money left. We use it to buy pairs that cost $t + 1$, as many as we can.

In the example, the pairs that cost at most 5 are 1, 2, 3, 3, 4, 4, 5, and their total is 22. If we also take the two pairs that cost 6, the total is 34, which is too much. So $t = 5$ and we have 8 dollars left. This is enough for one pair of cost 6, so the answer is again 8 items and 2 dollars.

2.5 Correctness

Greedy. Suppose the greedy rule buys k items. These are the k cheapest pairs. Any other k pairs cost at least as much, so no plan with k items has more money left. And $k+1$ items are not possible, because even the $k+1$ cheapest pairs together cost more than D. That is why greedy stopped.

Heap method. We need the heap to always have the cheapest unused pair in it. For any unused pair, the first unused pair of the same item is not more expensive, and it is in the heap if this item has been started. If the item has not been started, look at the last item that has been started. Its first pair is still in the heap (otherwise the next item would have been started too), and it is not more expensive, because that item is cheaper and its first pair uses the largest coupon. So the top of the heap is always the cheapest unused pair. Each pair is put into the heap only once, so we never buy a pair twice.

Binary search method. The pairs with cost at most t are the cheapest pairs, and we can pay for all of them, so greedy would buy them too. After that, the cheapest pairs left cost $t+1$. We can not pay for all of them (otherwise the search would have found $t+1$), so greedy buys as many as the money allows and then stops. This is what step 3 does.

3 Testing Results

3.1 Method

We tested on Linux with an Intel Xeon 2.10 GHz CPU, GCC 13.3.0 and Python 3.13. We compile with `-O2`, and also once with AddressSanitizer and UndefinedBehaviorSanitizer to find memory errors.

Our program can run any of the three methods on the same input, and they must print exactly the same line. We also wrote two small Python checkers. One tries all subsets of pairs and takes the best, so it does not depend on the greedy idea, but it is only possible for $N \leq 3$. The other one sorts all pairs in Python and is used for larger inputs.

3.2 Correctness Tests

Test group (count)	Result	Purpose
Hand-made cases (12)	PASS	The sample, and small cases we solved by hand: only one pair, not enough money for anything, exactly enough money for all pairs, one dollar short, all costs equal, $D = 1$, and values near 10^9.
Tiny inputs (500)	PASS	$N \leq 3$, compared with trying all subsets. This tests if the greedy idea itself is right.
Random small inputs (1,000)	PASS	$N \leq 40$, compared with the Python sort. Some have many equal costs, some have very large values.
Generated inputs (96)	PASS	Our four data types with N from 1 to 300 and D from 1 to 10^6, compared with the Python sort.
Large inputs (11)	PASS	$N = 4000$ with all three methods, and $N = 10^5$, $D = 10^6$ (the largest input allowed) with the two fast methods. They must agree.
Bad inputs (4)	PASS	Empty or broken input must give an error message, not a wrong answer.
Generator check (16)	PASS	The generator must give the same data on every compiler. We compare it with a Python version.

All 1,639 cases pass. With the sanitizers we run a smaller set of 354 cases and there is no error.

3.3 Test Data

For large inputs we wrote a generator, `code/gen.cpp`. It takes N, D, a type and a seed. The four types are:

1. **random**: coupons from 1 to 10^6 and prices from $10^6 + 1$ to 2×10^6. Most pairs are expensive, so only a few thousand items can be bought.
2. **close**: prices and coupons are close to each other, so the costs are between 1 and about 4,000 and many items can be bought.
3. **equal**: all prices are the same and all coupons are the same, so every pair costs 1 dollar. With $D = 10^6$ we buy 10^6 items, which is the most possible.
4. **far**: every pair costs more than 10^6, so nothing can be bought.

3.4 Timing Results

For timing, the program runs one method once to warm up and then 10 more times, and takes the median. We only time the method itself (with sorting), not reading the input. For every setting we use three inputs with different seeds and report the median of the three, in milliseconds. In the figures the error bars show the fastest and the slowest of the three. We do not time the far type because nothing happens there.

Experiment 1: more items. We fix $D = 10^6$ and change N from 250 to 10^5. Brute force is only run up to $N = 4000$. The last column shows how many items are bought when $N = 10^5$.

Type	Method	$N = 1000$	$N = 4000$	$N = 10^5$	Items
random	Brute force	48.9	1,112	–	
random	Heap	0.066	0.352	12.3	2,442
random	Binary search	0.071	0.436	13.8	2,442
close	Brute force	36.4	489	–	
close	Heap	0.389	0.890	12.5	135,508
close	Binary search	0.073	0.359	10.5	135,508
equal	Brute force	11.3	304	–	
equal	Heap	16.2	17.1	25.5	1,000,000
equal	Binary search	0.014	0.045	2.43	1,000,000

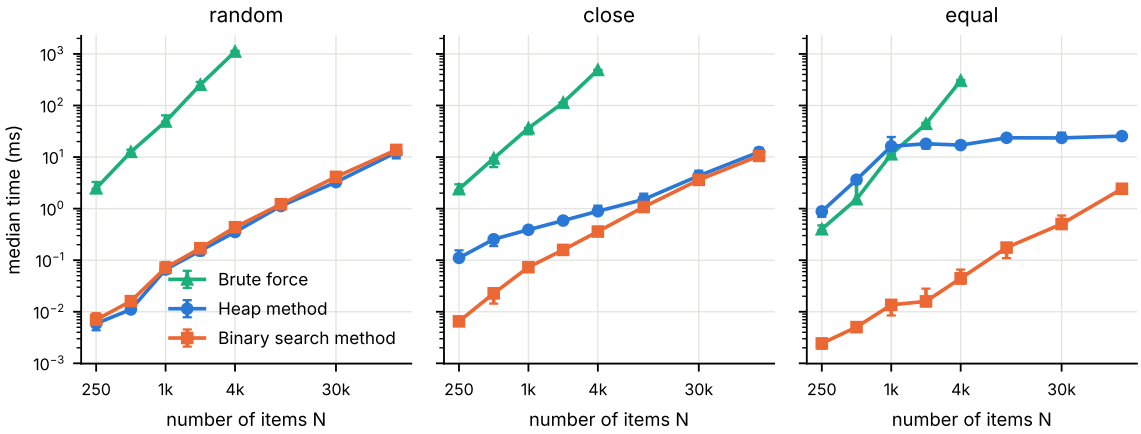


Figure 1: Experiment 1: time for different numbers of items, D = 1,000,000

Experiment 2: more money. We fix $N = 10^5$ and change D from 10^3 to 10^7. The value 10^7 is larger than the problem allows; we add it to see what happens when even more items are bought. The ”items” rows show the number of items bought.

Type	Row	$D = 10^4$	$D = 10^5$	$D = 10^6$	$D = 10^7$
close	items	6,102	28,446	135,508	646,022
close	Heap	9.69	8.12	13.1	29.1
close	Binary search	10.4	8.39	12.8	10.7
equal	items	10,000	100,000	1,000,000	10,000,000
equal	Heap	2.17	3.41	27.5	293
equal	Binary search	2.28	2.65	1.90	1.90

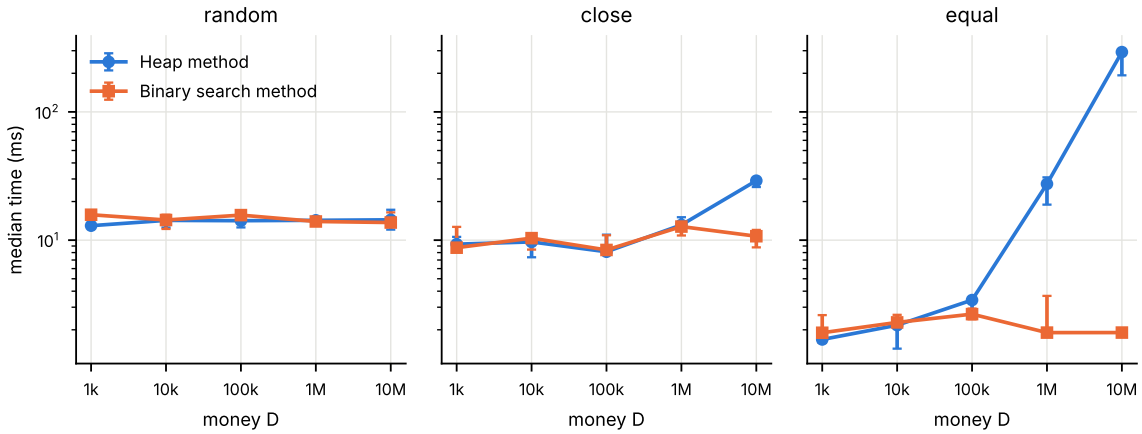


Figure 2: Experiment 2: time for different amounts of money, N = 100,000

4 Analysis and Comments

4.1 Time Complexity

Let K be the number of items we buy. Since every item costs at least 1 dollar, $K \leq D$.

Brute force. There are N^2 pairs and we sort them, so the time is $O(N^2 \log N)$.

Heap method. Sorting the prices and coupons takes $O(N \log N)$. Then every purchase does one pop and at most two pushes on the heap. The heap has at most one pair per item, so each operation is $O(\log N)$ [1]. The total is $O(N \log N + K \log N)$.

Binary search method. Sorting is also $O(N \log N)$. Counting for one t is one pass over the items, so $O(N)$. The binary search tries about $\log D$ values of t, which is about 20 when $D = 10^6$. The total is $O(N \log N + N \log D)$, and K does not appear in it.

4.2 Space Complexity

Brute force stores all pairs, so it needs $O(N^2)$ space. The heap method stores the two sorted arrays and a heap with at most N pairs, so $O(N)$. The binary search method stores the two arrays and the prefix sums, so also $O(N)$.

4.3 Discussion of Test Results

Brute force gets slow very quickly. In Experiment 1, when N doubles its time becomes 4 to 5 times longer, and at $N = 4000$ it already needs more than 1 second on random data. The other two methods need less than 1 ms there.

On random data the two fast methods take almost the same time, about 12 to 14 ms when $N = 10^5$. Only 2,442 items are bought, so most of the time is just sorting, which both methods do.

The equal type is where they differ. Here we buy 10^6 items, so the heap method has to do 10^6 rounds and takes about 16 to 26 ms. The binary search method takes only 2.4 ms at $N = 10^5$, which is about 10 times faster. In Experiment 2, when D grows from 10^6 to 10^7, the heap method goes from 27.5 ms to 293 ms, about 10 times more, the same as the number of items. The binary search method stays at about 2 ms, so it is about 150 times faster at $D = 10^7$. The close type is in the middle: the heap method only becomes slower at $D = 10^7$.

For the sizes in this problem both methods are fast enough, the slowest case is about 30 ms. We use the heap method as the default because it is shorter and it is exactly the greedy rule. The binary search method is better if the money could be much larger.

4.4 Limitations and Improvements

1. The heap method gets slower when more items are bought. If D were much larger than 10^6, the binary search method should be used.
2. We only measured on one machine with three inputs for each setting. On most inputs the sorting takes most of the time, so small differences between the two fast methods do not mean much.

5 Conclusion

To buy the most items we should always buy the cheapest (item, coupon) pair. Sorting all pairs works but only for small inputs. If we sort the prices and the coupons first, a min-heap can give us the cheapest pairs one by one, and a binary search on the cost can find the answer without going through the pairs. All three methods agree in our tests. The heap method is simple and fast enough for this problem, and the binary search method is faster when a lot of items can be bought.

6 References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022, ch. 6. Publisher.

7 Source Code Appendix

The source code is in the `code/` folder of the submission. `code/main.cpp` is the solver: `solve_brute`, `solve_heap` and `solve_search` are the three methods, and `main` reads the input and runs the chosen method. `code/gen.cpp` is the test data generator. The `README.md` shows how to compile and run them.